Les API

L'API GitHub

Table des matières

1. Enoncé 1
2. Création du token Github 1
3. Création d'un dépôt privé 2
4. Visualisation de son compte GitHub 3
5. Visualisation des dépôts publics 3
6. Visualisation des dépôts public et privés 3
7. Visualisation des informations sur un dépôt 5
8. Création d'un dépôt 5
9. Consultation du dépôt 6
10. Mise à jour d'un dépôt 6
11. Suppression d'un dépôt 7
12. Gestion depuis une application Web 8
    12.1. cUrl en PHP 8
    12.2. Le trait HttpClient (répertoire ClasseTechnique) 10
    12.3. La classe GitHub 11
    12.4. liste des dépôts publics et privés 12
    12
    12.1. La création d'un dépôt 13
    12.2. La mise à jour d'un dépôt 14

Guy Verghote

Les API : L'API GitHub

Objectif de l'activité professionnelle

A la fin de cette activité, vous serez capable :

D'utiliser les API fournies par Github permettant d'assurer la gestion des référentiels Savoir distinguer les différentes méthodes http. Mettre en place la phase d'authentification

1. Enoncé

Vous êtes chargé de mettre en œuvre l'API GitHub afin de gérer les opérations CRUD sur les référentiels.

Les opérations CRUD (Create, Read, Update, Delete) sont des opérations de base pour la gestion des données dans une base de données ou un système de gestion de contenu. Elles permettent de créer, lire, mettre à jour et supprimer des enregistrements.

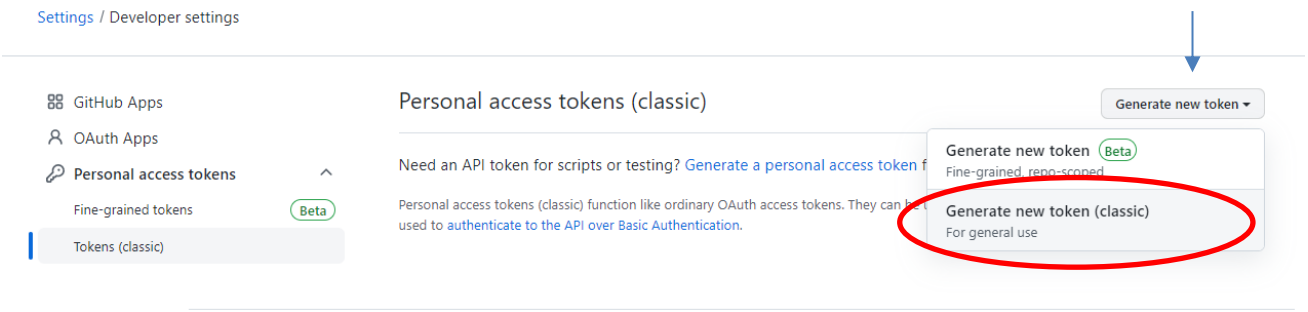
Les opérations CRUD sont réalisées dans une API par les méthodes HTTP suivantes :

Create (Créer) : POST :
Read (Lire) : GET :
Update (Mettre à jour) : PUT (Remplace complètement l'enregistrement existant) ou PATCH (Met à jour partiellement l'enregistrement existant).
Delete (Supprimer) : DELETE

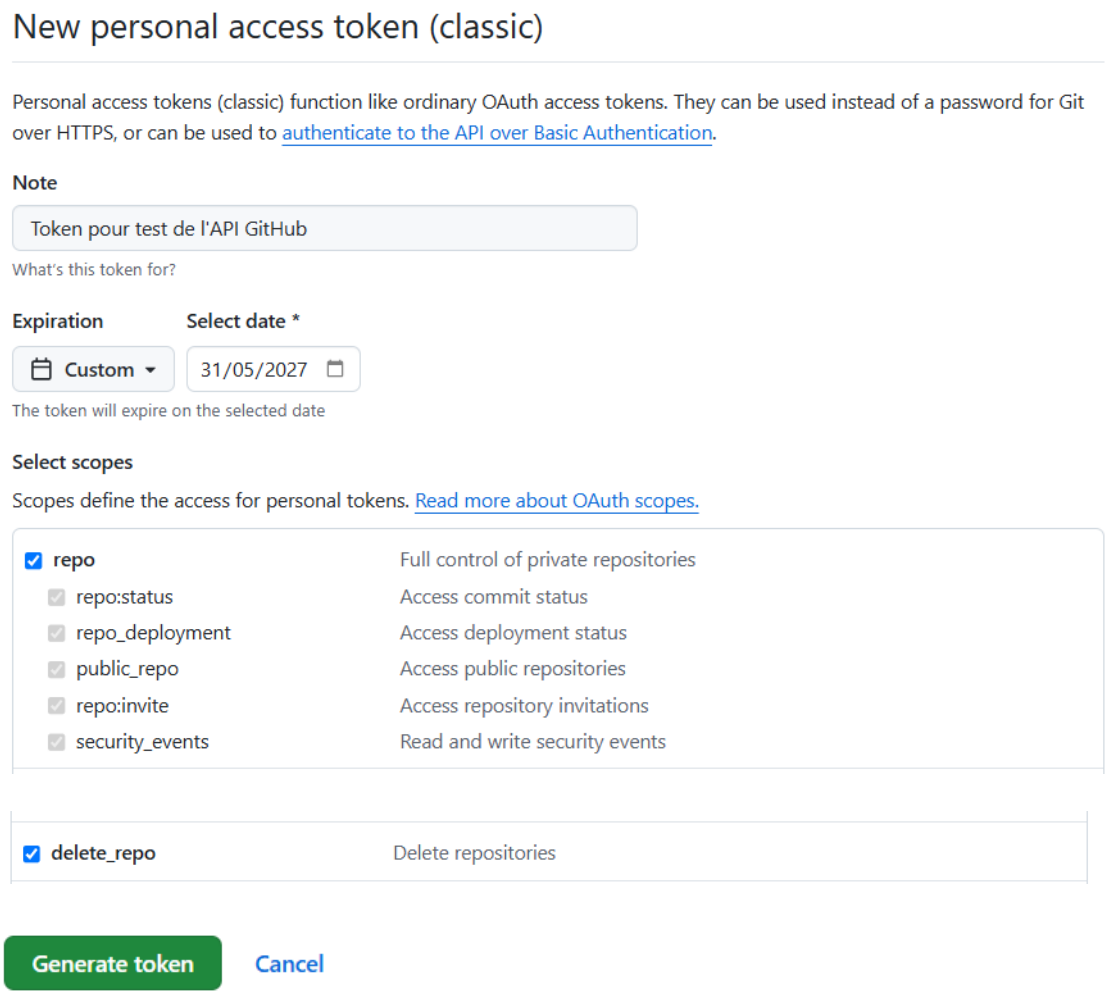
2. Création du token Github

Afin de pouvoir utiliser cette API, il faut commencer par définir un token sur GitHub a**fin de pouvoir utiliser les API nécessitant une authentification.**

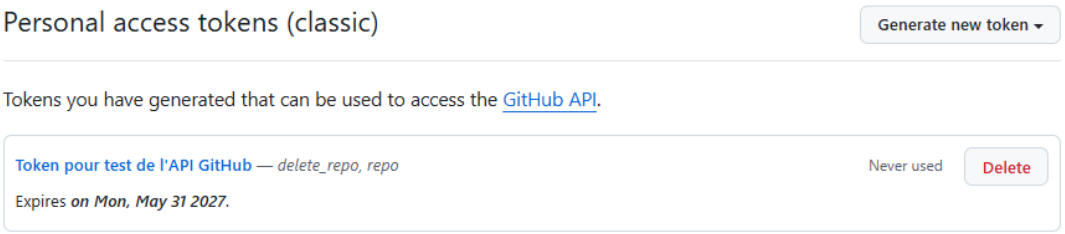
https://github.com/settings/tokens
(Settings – Développer settings – Personal access tokens classic)



Vous devrez confirmer votre identité
Au niveau de l'interface, **cocher les cases repo et delete_repo**

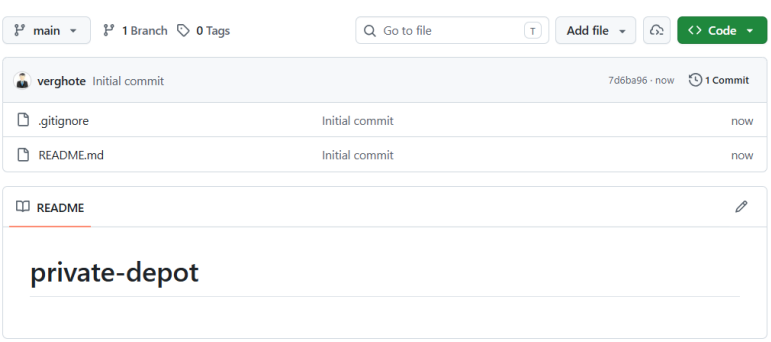


La clé apparait une seule fois, copier la dans votre coffre-fort KeePassXC



3. Création d'un dépôt privé

Nous avons besoin, pour nos tests de créer un dépôt privé 'private-depot'



4. Visualisation de son compte GitHub

Depuis le navigateur il est possible de consulter les informations d'un compte GitHub :

```
https://api.github.com/users/verghote
```

Nous obtenons le même résultat avec l'outil de test d'API Insomnia

Insomnia offre l'avantage de pouvoir visualiser le code dans les différents langages de programmation : PHP, Javascript, JQuery, C#, etc.

Ajouter le dossier GitHub
Créer une requête GET 'Mon compte' vers https://api.github.com/users/[nomCompte]

Nous pouvons aussi utiliser la commande curl sous cmder ou le terminal de PhpStorm

```
curl.exe --location https://api.github.com/users/verghote  (--location n'est pas obligatoire)
```

Si la requête n'est pas valide

```
{
  "message": "Not Found",
  "documentation_url": "https://docs.github.com/rest",
  "status": "404"
}
```

5. Visualisation des dépôts publics

Depuis le navigateur :

```
https://api.github.com/users/verghote/repos
```

En utilisant l'outil de test d'API Insomnia

Créer une requête GET 'Mes dépôts publics'
Url : https://api.github.com/users/ [nomCompte]/repos

En utilisant la commande curl sous cmder ou le terminal de PhpStorm

```
curl.exe https://api.github.com/users/verghote/repos
```

6. Visualisation des dépôts public et privés

Documentation :
https://docs.github.com/en/rest/repos/repos?apiVersion=2022-11-28#list-public-repositories
https://docs.github.com/rest/repos/repos#list-repositories-for-the-authenticated-user

Depuis la mise en place de l'authentification renforcée sur l'API GitHub, il est obligatoire d'utiliser un token personnel (PAT) pour interroger l'API et accéder aux dépôts, notamment les dépôts privés.

Il n'est donc plus possible d'effectuer cette requête directement depuis un navigateur, car celui-ci ne permet pas d'ajouter un Bearer Token dans les en-têtes.

L'url est la suivante : https://api.github.com**/user/repos?affiliation=owner**

Cette URL retourne tous les dépôts publics et privés de l'utilisateur authentifié

Il devient nécessaire de s'authentifier en ajoutant les paramètres d'autorisation au niveau de la requête. Sinon la réponse obtenue sera :


```
{
"message": "Requires authentication",
"documentation_url": "https://docs.github.com/rest/repos/repos#list-repositories-for-the-authenticated-user",
"status": "401"
}
```


Nous utilisons l'outil Insomnia pour cela :

Créer une requête GET 'Mes dépôts publics et pricés'
Url : https://api.github.com/users/ [nomCompte]/repos

Dans l'onglet Auth
Type : Bearer Token
TOKEN : <votre token>

Si token non valide


```
{
    "message": "Bad credentials",
    "documentation_url": "https://docs.github.com/rest"
}
```


Remarque : Insomnia **génère automatiquement l'en-tête HTTP Authorization**. Cela évite de l'écrire soi-même dans l'onglet Headers (Key : Authorization, value : Bearer ghp_...)
GitHub accepte encore l'ancien type token à la place de Bearer

En utilisant la commande curl sous cmder ou le terminal de PhpStorm

```
curl --location "https://api.github.com/user/repos" --header "Authorization: Bearer ghp_..."
```

Le Bearer est un standard RFC 6750 issu d'OAuth 2.0.
Cela peut être un JWT, un token opaque (comme ceux de GitHub), ou même un token chiffré.

7. Visualisation des informations sur un dépôt

L'url est la suivante : https://api.github.com**/repos/nomUser/nomDepot**

L'authentification n'est pas nécessaire pour un dépôt public.
Pour un dépôt privé, sans authentification ou un dépôt inexistant la réponse est la suivante :


```
{
  "message": "Not Found",
  "documentation_url": "https://docs.github.com/rest/repos/repos#get-a-repository",
  "status": "404"
}
```


Depuis Insomnia
Créer une nouvelle requête GET 'Consultation dépôt' dans le dossier GitHub
url : https://api.github.com/repos/*verghote*/private-depot
Dans l'onglet authorization : Type : Bearer Token - TOKEN : <votre token>

8. Création d'un dépôt

Documentation
https://docs.github.com/en/rest/reference/repos#create-a-repository-for-the-authenticated-user

GitHub attend **obligatoirement un corps JSON**

Sous Insomnia
Ajouter une requête "Création dépôt"
URL : **https://api.github.com/user/repos**
Méthode : **POST**
Authentification nécessaire en ajoutant votre token dans Auth

Pour passer les propriétés attendues dans le Body, il faut choisir le format JSON

```
{
   "name": "monPremierDepot",
   "description": "Formation"
}
```

D'autres propriétés peuvent être transmises par exemple "private"
Par défaut un dépôt est public ("private" : false)
En choisissant JSON, Insomnia ajoute automatiquement l'en-tête Content-Type: application/json.

Les accents et espaces dans le nom sont automatiquement remplacés par des tirets

Résultat attendu : 201 Created


```
{
    "id": 603443792,
    "node_id": "R_kgDOI_fSUA",
    "name": "monPremierDepot",
    "full_name": "verghote/monPremierDepot",
    "private": false,
    "owner": {
        "login": "verghote",
        "id": 102029585,
        "node_id": "U_kgDOBhTZEQ",
        "avatar_url": "https://avatars.githubusercontent.com/u/102029585?v=4",
        "gravatar_id": "",
        "url": "https://api.github.com/users/verghote",
```

Si le dépôt existe déjà : 422 Unproccessable Entity


```
{
    "message": "Repository creation failed.",
    "errors": [
        {
            "resource": "Repository",
            "code": "custom",
            "field": "name",
            "message": "name already exists on this account"
        }
    ],
    "documentation_url": "https://docs.github.com/rest/repos/
        repos#create-a-repository-for-the-authenticated-user",
    "status": "422"
}
```


9. Consultation du dépôt

Vous pouvez réutiliser votre requête "Consultation dépôt" pour vérifier l'existence de ce dépôt.

10. Mise à jour d'un dépôt

Documentation : https://docs.github.com/en/rest/reference/repos#update-a-repository

La mise à jour s'effectue à l'aide d'une requête utilisant la méthode PATCH
L'url est la suivante : https://api.github.com/repos/**<user>/<repo>**
L'objectif est de rendre le dépôt 'monPremier privé.

Sous Insomnia
Ajouter une requête "Mise à jour dépôtt"
URL : **https://api.github.com/repos/*verghote*/monPremierDepot**
Méthode : **PATCH**
Authentification nécessaire en ajoutant votre token dans Auth

Résultat attendu : **200 Ok**

Si le body n'est pas renseigné : **400** Bad Request


```
{
    "message": "Body should be a JSON object",
    "documentation_url": "https://docs.github.com/rest/repos/repos#update-a-repository"
}
```


Si le body n'est pas au format JSON : **400** Bad Request


```
{
    "message": "Problems parsing JSON",
    "documentation_url": "https://docs.github.com/rest/repos/repos#update-a-repository"
}
```

11. Suppression d'un dépôt

Documentation : https://docs.github.com/en/rest/reference/repos#delete-a-repository

La suppression s'effectue à l'aide d'une requête utilisant la méthode DELETE
URI : https://api.github.com/repos/**<user>/<repos>**
Réponse : 204 No Content

Sous Insomnia
Ajouter une requête "Suppression dépôt"
URL : **https://api.github.com/repos/*verghote*/monPremierDepot**
Méthode : **PATCH**
Authentification nécessaire en ajoutant votre token dans Auth

Résultat attendu : **204** No Content

En cas d'absence d'authentification une erreur 403 est retournée :


```
{
    "message": "Must have admin rights to Repository.",
    "documentation_url": "https://docs.github.com/rest/reference/repos#delete-a-repository"
}
```


En cas de token non valide une erreur 401 est retournée


```
{  "message": "Bad credentials",
   "documentation_url": "https://docs.github.com/rest",
   "status": "401"
}
```


Si le dépôt n'existe pas **404** Not Found


```
{
  "message": "Not Found",
  "documentation_url": "https://docs.github.com/rest/repos/repos#delete-a-repository",
   "status": "404"
}
```


Si le dépôt a été renommé et que la command curl n'utilise pas l'option -L


```
{  "message": "Moved Permanently",
   "url": "https://api.github.com/repositories/881204802",
   "documentation_url": "https://docs.github.com/rest/guides/best-practices-for-using-the-rest-api#follow-redirects"
}
```

12. Gestion depuis une application Web

Nous allons refaire les mêmes actions à partir d'une application Web.

12.1. cUrl en PHP

L'extension cURL (Client URL Library) permet à PHP d'envoyer des requêtes HTTP/HTTPS vers des API ou des serveurs distants. Elle est particulièrement utile pour consommer des API REST, comme celle de GitHub.

cURL :
établit une connexion HTTP(S),
envoie une requête (GET, POST, PATCH, DELETE, etc.),
lit la réponse (souvent en JSON),
et gère les erreurs réseau ou applicatives.

Exemple simple : requête GET sur l'API GitHub

```
$url = "https://api.github.com/users/nomUser/repos";
// 1. Initialisation d'une session cURL
$ch = curl_init();
// 2. URL cible
curl_setopt($ch, CURLOPT_URL, $url);
// 3. Retour du résultat dans une variable (et non affichage direct)
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
// 4. Entête user_agent obligatoire pour GitHub API
curl_setopt($ch, CURLOPT_USERAGENT, 'MyApp');
// 5. Chemin vers le fichier de certificat
curl_setopt($ch, CURLOPT_CAINFO, 'j:/cacert.pem');
// 6. Exécution de la requête
$response = curl_exec($ch);
// 7. Code HTTP (200, 404, 401, etc.)
$code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
// 8. Fermeture de la session
curl_close($ch);
// 9. Décodage JSON en tableau associatif
$data = json_decode($response, true);
```

GitHub impose l'indication d'un User-Agent dans l'entête de la requête. En cas d'absence une erreur 403 est renvoyée :

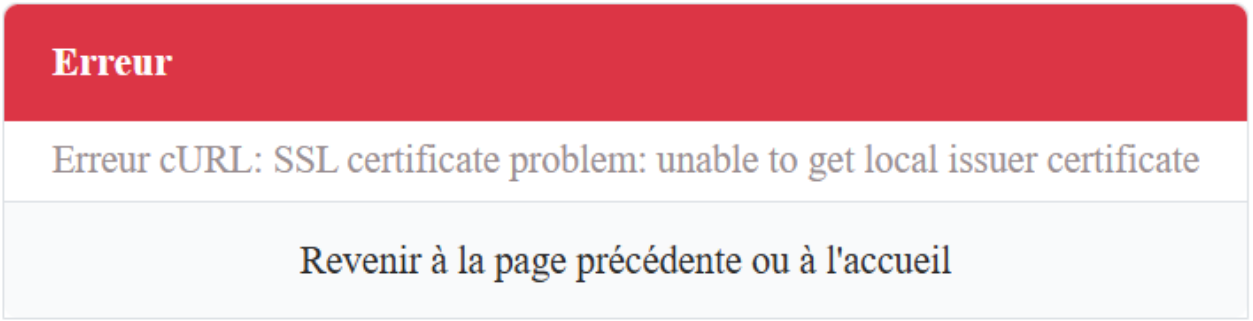
```
Demande interdite par les règles administratives. Veuillez-vous assurer que votre demande comporte un en-tête User-Agent.
```

Il recommande d'utiliser votre nom d'utilisateur GitHub ou le nom de votre application pour la valeur d'en-tête User-Agent. Cela permet à GitHub de vous contacter en cas de problème.

Le paramètre CURLOPT_CAINFO => 'j:/cacert.pem'
Cette option permet de spécifier explicitement un fichier de certificats racines pour la validation.
Ce paramètre est particulièrement nécessaire dans les environnements où cURL ne trouve pas automatiquement les certificats CA ou lorsque vous vous connectez à un serveur avec un certificat SSL auto-signé ou un certificat provenant d'une autorité de certification non reconnue par défaut.

Plus précisément, cela signifie que cURL ne parvient pas à vérifier le certificat SSL du serveur car il ne peut pas trouver l'autorité de certification (CA) locale nécessaire pour vérifier la chaîne de certificats.

En l'absence de cette ligne la redirection vers la page d'erreur retourne le message suivant :



Si une authentification est nécessaire il faut ajouter le paramètre CURLOPT_HTTPHEADER :

```
$headers = [
    'Authorization: token ghp_XXXXX',
    'Accept: application/vnd.github+json',
];

curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
```

Si vous êtes dans un environnement de développement et que vous ne souhaitez pas valider le certificat SSL du serveur (bien que cela ne soit pas recommandé pour des raisons de sécurité), vous pouvez désactiver la vérification SSL : **CURLOPT_SSL_VERIFYPEER** => false
Cela expose votre connexion à des attaques de type "man-in-the-middle" et ne devrait être utilisé qu'à des fins de test ou dans des environnements contrôlés.

Remarque :

En règle générale, il est possible d'utiliser la fonction PHP file_get_content($url) pour récupérer directement la réponse d'un API de type GET mais il y a certaines limitations et conditions à prendre en compte

Si vous avez besoin de personnaliser l'en-tête HTTP de la requête (par exemple, pour ajouter des clés API ou des paramètres), vous pouvez utiliser le contexte de flux (stream context).

```
// version avec file_get_contents
// Récupérer le token d'autorisation depuis la variable d'environnement
$token = getenv('GITHUB_TOKEN');
$options = [
    'http' => [
        'header' => "Authorization: Bearer $token\r\n" .
                    "User-Agent: PHP\r\n"
    ]
];
$context = stream_context_create($options);
$data = file_get_contents($url, false, $context);
```

12.2. Le trait HttpClient (répertoire ClasseTechnique)

Pour interroger une API de manière propre et éviter d'écrire du code cURL dupliqué dans une application cliente, nous avons mis en place un Trait PHP nommé HttpClient.

Qu'est-ce qu'un Trait en PHP ?
Contrairement à une classe classique, un trait ne peut pas être instancié directement. Il s'agit d'un « paquet » de méthodes et de propriétés que l'on peut incorporer à l'intérieur d'une classe. Cela permet de partager du code entre plusieurs classes sans utiliser l'héritage multiple (interdit en PHP).

Le trait HttpClient centralise toute la logique technique de communication HTTP avec l'API (gestion des en-têtes, des méthodes, du jeton Bearer, des erreurs cURL et du décodage JSON).

Analyse du fonctionnement de la méthode requete()
La méthode centrale du trait est la méthode protégée requete().
Voici comment elle orchestre l'appel cURL :

Vérification de l'URL de base : Elle s'assure que l'URL du serveur distant a bien été configurée.

Construction des en-têtes (Headers) :

Par défaut, elle force l'utilisation du format JSON (Accept et Content-Type).

Si un jeton d'authentification ($token) est fourni, elle ajoute automatiquement l'en-tête Authorization: Bearer <token>.

Préparation de la requête cURL (curl_init et curl_setopt_array) :

Concaténation propre de l'URL de base et de l'endpoint demandé.

Configuration de la méthode HTTP (GET, POST, PATCH, DELETE).

Envoi des données ($data) transformées en chaîne JSON (json_encode) pour les méthodes d'écriture.

Exécution et gestion des erreurs techniques :

curl_exec() exécute la requête. Si le serveur distant est injoignable, une exception UserException est levée.

curl_getinfo() récupère le code de statut HTTP retourné par l'API (ex: 200, 201, 401, 404, etc.).

Analyse de la réponse et gestion des erreurs métiers :

Le corps de la réponse JSON est décodé en tableau associatif PHP (json_decode).

Si le code HTTP indique une erreur (< 200 ou >= 300), le code extrait intelligemment le message d'erreur renvoyé par l'API (qu'il soit dans un tableau errors, error ou via un libellé HTTP standard) et lève une exception contenant ce message et le code HTTP.

Si tout est correct, elle retourne directement les données sous forme de tableau PHP exploitable par l'application cliente.

12.3. La classe GitHub

La classe métier GitHub utilise le trait HttpClient. Les méthodes applicables sur un objet GitHub permettent de réaliser l'ensemble des opérations de gestion sur ses propres dépôts.

Le constructeur instancie le token et le nom de l'utilisateur en les récupérant de puis des variables d'environnement

```
class GitHub {
    use HttpClient;

    private string $token;
    private string $username;
    private string $apiUrl = 'https://api.github.com/';

    public function __construct() {
        $this->token = getenv('GITHUB_TOKEN');
        $this->username = getenv('GITHUB_USERNAME');
    }
```

Pour éviter l'écriture du token dans le code, il faut utiliser une **variable d'environnement**.
La variable d'environnement est déclarée au niveau du serveur Apache en utilisant le fichier .htaccess (pour la démonstration, normalement cela se fait dans le fichier de configuration d'Apache httpd.conf)

```
SetEnv GITHUB_TOKEN ghp_....
SetEnv GITHUB_USERNAME …..
```

1. Compléter le fichier .htaccess du répertoire github en y plaçant votre token personnel.

Toutes les API du web ne respectent pas strictement les mêmes conventions. L'API de GitHub possède ses propres exigences techniques que notre méthode générique requete() du trait ne peut pas deviner par défaut.

Pour éviter de dupliquer du code dans chaque méthode (getAll, ajouter, modifier, etc.), nous avons créé une méthode intermédiaire privée requeteGitHub(). Elle fait office de "passerelle sur mesure" :

```
private function requeteGitHub(string $method, string $endpoint = '', ?array $data = null): array {
    $headers = [
        'User-Agent: ' . $this->username,
        'Authorization: token ' . $this->token, // GitHub utilise le préfixe 'token' historique
        'Accept: application/vnd.github+json',
    ];

    return $this->requete($method, $endpoint, $data, null, $headers);
}
```

Cette méthode définit les en-têtes obligatoires exigés par GitHub :

User-Agent : GitHub bloque systématiquement (erreur 403) tout client qui ne s'identifie pas avec un nom d'utilisateur ou un nom d'application.

Authorization : GitHub utilise un format de jeton spécifique avec le préfixe token (ex: Authorization: token ghp_xxxxx).

Accept : Elle précise la version et le format de données attendus (application/vnd.github+json).

Elle transmet ensuite ces en-têtes personnalisés à la méthode requete() de notre trait HttpClient, qui se charge de l'exécution cURL brute.

Grâce à cette conception, toutes les méthodes métier de la classe (getAll, ajouter, modifier, supprimer) n'ont plus qu'à appeler requeteGitHub() l'esprit léger, en se concentrant uniquement sur l'URL de l'endpoint et les données à transmettre !

2. Etudier les méthodes de la classe Github

12.4. liste des dépôts publics et privés



Mes Dépôts GitHub

Nom	Type	Description
api	PRIVÉ	Application web mettant en oeuvre les API
bloc2-evaluation	PUBLIC	Aucune description
bloc3-etude-de-cas	PUBLIC	Liste des études de cas préparant à l'épreuve E7
consultation	PUBLIC	Application web mettant en oeuvre la consultation des données contenues dans les tables d'une base de données
documentation-obsidian	PUBLIC	Documentation concernant les blocs 2 et 3
documentation-ppe-vds	PUBLIC	Cahier des charges - modules à développer
evaluation	PUBLIC	Aucune description
gestion	PUBLIC	Application web mettant en oeuvre les opérations de mise à jour et de suppression de données au niveau des tables d'une base de données
immo-3000	PUBLIC	Aucune description
inscription	PUBLIC	Aucune description

La méthode **getAll()** de la classe GitHub doit permettre de récupérer l'ensemble des dépôt publics et privés. Elle doit pour cela lancer la requête GET :

***https://api.github.com*/user/repos?affiliation=owner**

Elle fait appel à la méthode executer() de la classe Github pour lancer la requête

3. Compléter la méthode getAll()

Le script **github/index.php** doit instancier un objet GitHub, puis appeler la méthode getAll() afin d'envoyer les données encodées en JSON côté client

Le script index.js affiche les données en parcourant le tableau lesDepots. La colonne description est masquée sur les petits écrans.

4. Compléter le script index.php et réaliser le test fonctionnel.

12.1. La création d'un dépôt



La **méthode ajouter()** de la classe GitGub doit permettre d'ajouter un nouveau dépôt, en précisant son nom, sa description et si le dépôt est privé (true/false).
Elle doit pour cela lancer la requête ***https://api.github.com/*user/repos en utilisant la méthode POST et en passant les trois paramètres (nom, description et type) dans un tableau associatif $data.**

5. Compléter la méthode ajouter()

Le script index.php n'a rien à charger.
Le script **ajax/ajouter.php** doit :

- Vérifier la transmission des paramètres POST
- Récupérer des paramètres transmis
- Vérifier que le paramètre 'private' est bien égale à 1 ou 0, le mettre 0 sinon
- Réaliser l'ajout en créant un objet GitHub et en appelant la méthode ajouter
- Analyser la réponse obtenue : si elle contient une clé 'error', il faut renvoyer le message d'erreur sinon il faut envoyer, encodée en JSON une clé 'success'

6. Compléter le script ajax/ajouter.php et réaliser les tests fonctionnels sous Insomnia

Attention sous Insomnia : ne pas oublier d'ajouter l'entête X-Requested-With: 'XMLHttpRequest'

En cas d'échec

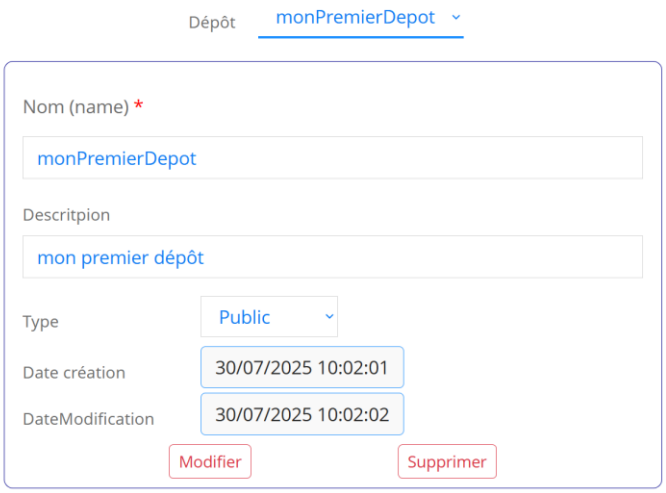
```
{
    "error": {
        "global": "name already exists on this account"
    }
}
```

Le **script ajout.js** se charge de lancer le script 'ajax/ajouter.php' par un appel Ajax réalisé à l'aide de la function appelAjax().
En cas de succès, on retourne sur le script index.php après avoir afficher un message de réussite

```
success: (data) => retournerVersApresConfirmation(data.message, '../liste/')
```

7. Compléter le script ajout.js et réaliser les tests fonctionnels

12.2. La mise à jour d'un dépôt



Au niveau de la classe GitHub nous utilisons les méthodes :

- getLesNoms() qui retourne les noms des dépôts (permet de remplir la zone de liste).
- getByName() qui retourne les informations du dépôt passé en paramètre
- modifier() pour demander la modification,
- supprimer() pour demander la suppression

La **méthode getLesNoms()** permet de récupérer uniquement le nom de chaque dépôt.
Elle lance la même requête que la méthode getAll() mais parcourt la réponse afin d'en extraire uniquement les noms :

```
public function getLesNoms(): array    {
        $lesDepots = $this->executer("GET", "user/repos?affiliation=owner");
        $lesNoms = [];
        foreach ($lesDepots as $depot) {
            $lesNoms[] = $depot['name'];
        }
        return $lesNoms;
}
```

Il est possible de simplifier cette méthode en utilisant la fonction PHP array_column(tableu, colonne) qui ne renvoie les valeurs d'une seule colonne dans le tableau d'entrée

8. Réécrire en une seule instruction la méthode getLesNoms()

La **méthode getByName()** permet de récupérer l'ensemble des informations concernant un dépôt. Elle attend en paramètre le nom du dépôt concerné .
Elle doit pour cela lancer la requête **https://api.github.com/repos/*nomUser/nomDepot*** en utilisant la **méthode GET**

9. Compléter la méthode getbyName()

La **méthode modifier()** permet de modifier ces trois informations. Elle attend en paramètre le nom du dépôt concerné et un tableau associatif comportant les champs modifiés. La clé correspond au nom du champ et la valeur à la nouvelle valeur du champ.
Elle doit pour cela lancer la requête **https://api.github.com/repos/*nomUser/nomDepot*** en utilisant la **méthode PATCH**

10. Compléter la méthode modifier()

La **méthode supprimer()** permet de supprimer un dépôt. Elle attend en paramètre le nom du dépôt concerné.
Elle doit pour cela lancer la requête **https://api.github.com/repos/*nomUser/nomDepot*** en utilisant la **méthode DELETE**

11. Compléter la méthode supprimer()

Le script **github/maj.php** doit instancier un objet GitHub, puis appeler la méthode getlesNom() afin d'envoyer les données encodées en JSON côté client.

12. Compléter le script maj.php

Le script **github/ajax/getbyname.php** instancie un objet GitHub, puis appelle la méthode getlesNom() afin d'envoyer les données encodées en JSON côté client

Le script **ajax/getbyname.php** doit :

- Vérifier et récupérer le paramètre transmis dans la variable $name
- Créer un objet GitHub et appeler la méthode getByName
- Envoyer la réponse dans le format JSON

13. Compléter le script ajax/getbyname.php et réaliser les tests fonctionnels sous Insomnia

Le script **ajax/modifier.php** doit :

- Vérifier et récupérer des paramètres transmis dans les variables $name et $lesValeurs
- Réaliser la modification en créant un objet GitHub et en appelant la méthode modifier
- Analyser la réponse obtenue : si elle contient une clé 'error', il faut renvoyer le message d'erreur sinon il faut envoyer, encodée en JSON une clé 'success'

14. Compléter le script ajax/modifier.php et réaliser les tests fonctionnels sous Insomnia

Le script **ajax/supprimer.php** doit :

- Vérifier et récupérer le paramètre transmis dans la variable $name
- Réaliser la suppression en créant un objet GitHub et en appelant la méthode supprimer
- Analyser la réponse obtenue : si elle contient une clé 'error', il faut renvoyer le message d'erreur sinon il faut envoyer, encodée en JSON une clé 'success'

15.Compléter le script ajax/supprimer.php et réaliser les tests fonctionnels sous Insomnia

Il reste à compléter côté client le script maj/index.js, chargé de réaliser les appels Ajax des scripts côté serveur.

La fonction getDepot(name) réalise l'appel Ajax du script 'ajax/getbyname.php' afin de récupérer les données sur le dépôt dont le nom est passé en paramètre.
La fonction afficherReferentiel doit être appelée en cas de succès.

La fonction getDepot() est lancée au chargement, sur le changement de valeur de la zone de liste et dans les fonctions assurant la modification et la suppression

16. Compléter la fonction getDepot(name) du script maj.js

La fonction afficherReferentiel(data) affiche les données récupérées sur l'interface.
Les dates GitHub sont au format Gmt-0 "2026-10-22T03:26:41Z", il faut les convertir en français pour les avoir en Gmt-2

```
let date = new Date(data.created_at);
dateCreation.value = date.toLocaleString('fr-FR');
```

La fonction supprimer(name), appelée après confirmation du clic sur le bouton supprimer, doit réaliser l'appel Ajax du script ajax/supprimer.php. En cas de succès il faut mettre à jour l'interface en supprimant l'objet sélectionné dans la zone de liste et relancer la fonction getDepot(name) afin d'afficher les informations concernant le dépôt maintenant sélectionné

17. Compléter la fonction supprimer(name) du script maj.js

La fonction modifier(name), comme pour toute demande de modification, commence par récupérer dans le tableau lesValeurs, les propriétés ayant été modifiées (comparaison de la valeur sur le formulaire et celle conservée dans l'objet global depot)

Il faut ensuite réaliser l'appel Ajax du script ajax/modifier.php En cas de succès il faut mettre à jour l'interface au niveau de la zone de liste car le nom du dépôt peut avoir été modifié. Il faut aussi relancer la fonction getDepot(name) car la date de modification a été modifiée.

18.Compléter la fonction modifier(name) du script maj.js et réaliser les tests fonctionnels du module.